ENCODER-IMU ODOMETRY ROBOT

John Renner

ELEC6101: Autonomous Vehicles I

University of New Haven, Fall 2025

CONTENTS

1. OVERVIEW 2
2. BACKGROUND 2
3. METHODOLOGY 3
   3.2.1 HARDWARE ARCHITECTURE 3
   3.2.2 SOFTWARE 5
   3.2.3 SENSOR DATA ACQUISITION 5
   3.2.4 ODOMETRY AND POSE ESTIMATION 5
   3.2.5 CONTROL AND NAVIGATION 6
   3.2.6 COMMUNICATION 6
   3.2.7 EXPERIMENTAL TESTING 7
4. RESULTS 8
5. DISCUSSION 10
6. FUTURE IMPROVEMENTS 10
7. CONCLUSION 11
8. REFERENCES 11

1. OVERVIEW

The original goal of this project was to create an autonomous robot capable of identifying and navigating into an empty parking space. Early in the project, it became clear that reliable parking behavior would depend on accurate estimation of position and orientation. Initial attempts at position tracking using a 6-axis accelerometer/gyroscope yielded poor results, particularly for distance tracking. As a result, the scope of the project shifted away from parking related behaviors and instead focused on devising a more accurate method for tracking position.

The resulting robot uses wheel encoders to estimate distance and the accelerometer for heading estimation. Odometry and sensor processing are handled by a dedicated ESP32 that was added to the robot. The ESP32 communicates with the Arduino MCU and accelerometer breakout board via I2C, and a Bluetooth connection provides a command and diagnostic user interface. These sensor readings are used to support a wheel speed PID controller, as well as IOSFL for navigating to a target point.

This report describes the evolution of the project, including hardware modifications, software development, and experimental testing. While sensor readings and odometry show accuracy in controlled testing, mechanical issues with motor dead zones, wheel slip, and friction made it difficult for the robot to perform the fine corrections needed for precise navigation. The report concludes with suggestions for continued work on the project.

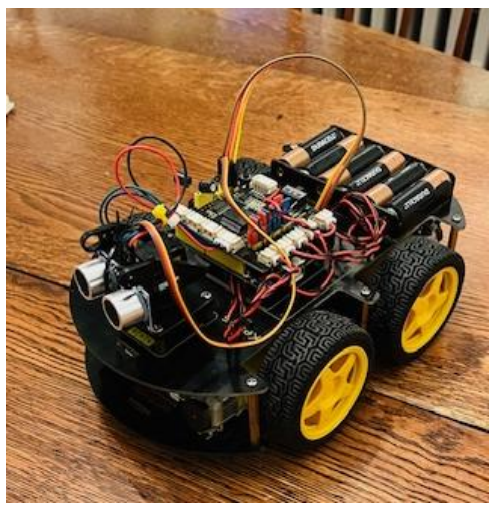

Figure 1. Robot

2. BACKGROUND

Accurate navigation of autonomous vehicles depends on reliable estimation of position and orientation. In indoor environments where external localization techniques such as GPS are unavailable, mobile robots rely on dead reckoning, which estimates position by integrating sensor readings over time starting from a known position. However, dead-reckoning performance is strongly influenced by sensor choice and any physical limitations of the platform.

Wheel encoders are among the sensors commonly used for navigating by dead reckoning. By measuring the rotation of a wheel with a known diameter, encoders can be used to measure wheel rotation. While

error can accumulate due to wheel slip, uneven traction, and mechanical tolerances, encoder based odometry remains a practical solution for navigation.

Inertial sensors, including accelerometers and gyroscopes, are also commonly used for navigation in mobile robots. While inertial sensors can provide useful motion information, acceleration based position estimation is particularly sensitive to noise and bias. This is due to the double integration of acceleration to calculate position, which can cause small errors to blow up. Because of this, accelerometers are more commonly used to estimate orientation through angular motion rather than straight-line position, unless combined with other sensors.

Beyond sensing, effective navigation requires strong control techniques. Low level control of wheel motion can be achieved through a PID controller, which regulates wheel speed based on sensor feedback. PID control is widely used due to its simplicity and effectiveness.

3. METHODOLOGY

3.2.1 HARDWARE ARCHITECTURE

The mobile robot was based on the Keyestudio 4WD V2 Robot. The original chassis and motor driver were kept. However, several hardware modifications were made to support more accurate sensing and control.

The stock DC motors were replaced with FIT0450 motors equipped with incremental encoders from DFRobot, producing 960 ticks per revolution of the drive shaft. The addition of the encoders significantly expanded the sensing capabilities of the robot by allowing wheel rotation to be measured directly. The replacement motors exhibited a different gearing ratio than the stock motors (1:48 for stock motors, 1:120 for the replacements).

Each encoder, when rotated, produces two pulse trains with a phase difference of 90 degrees on separate A and B channels. By counting edges while noting which channel leads in phase, the speed and direction of wheel rotation can be calculated. Because of the timing precision required to accurately count encoder ticks per control loop, it is preferred to trigger these encoder counts through interrupts rather than simple polling. Therefore, an MCU with at least 4 interrupt-capable GPIO pins must be used to read the encoder data.

The Keyestudio kit uses an Arduino Uno as it's MCU, with a dedicated motor driver shield sitting on top of the board. This poses two issues: 1.) the Arduino does not have enough interrupt-capable GPIO pins necessary for encoder reading, and 2.) very few input pins are available to interface with the Arduino through the motor driver shield. This motivated the decision to implement a second MCU capable of handling all of the necessary inputs. Then, high level processing could be offloaded onto this MCU, and the Uno could become a dedicated motor driver.

In the final configuration, an ESP32 was chosen for handling encoder inputs, IMU data processing, and odometry computation. The ESP32 communicates with the Arduino motor controller through I2C. The ESP32 was also chosen for its Bluetooth and WiFi capabilities, which would later be used to create the user interface for controlling the robot.

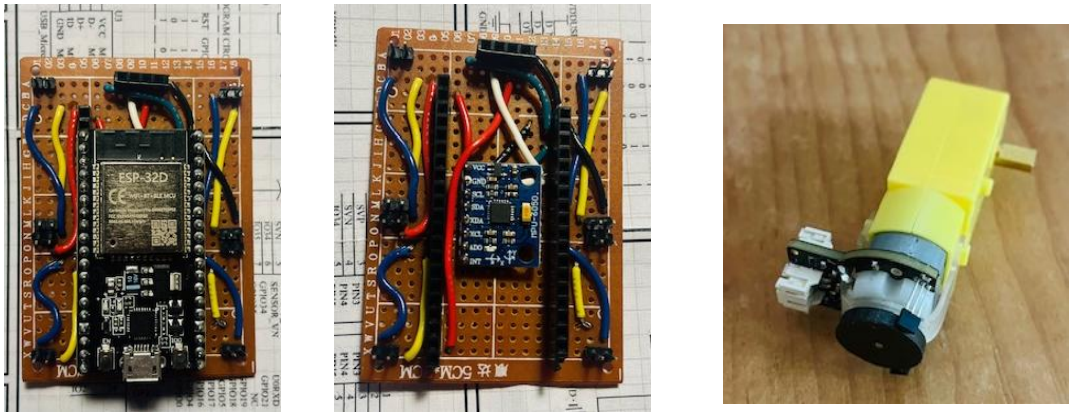

Figure 2. (Left to Right) ESP32 Breakout Board; ESP32 Breakout Board without ESP32 to show IMU mounting location; DFRobot FIT0450 Motor with Encoder

An MPU6050 clone breakout board (6-axis accelerometer and gyroscope IMU) was also mounted on the robot to provide inertial sensing. The IMU outputs linear and angular acceleration along all three axes. The IMU also comes equipped with an on-board digital low pass filter. The IMU communicates with the master MCU via I2C communication, so the LED display was removed from the Keyestudio kit to make pins A4 and A5 (SCL and SDA lines) available on the Keyestudio MCU.

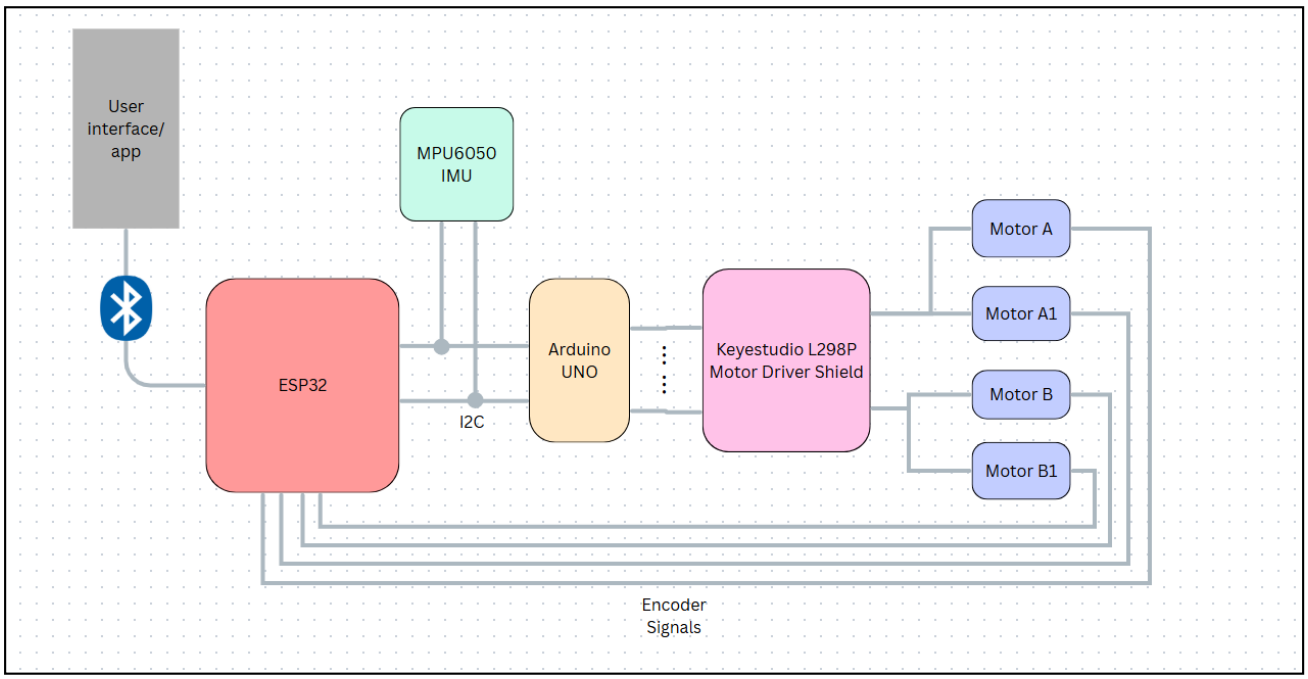

Figure 3. High Level System Diagram

3.2.2 SOFTWARE

The robot software was designed using a layered architecture that separates sensing, estimation, control, and communication across two microcontrollers. This structure enables sensor processing and state estimation to be handled separately from motor control, while maintain a clear, modular flow of information between systems.

At a high level, the ESP32 acts as the primary processing unit for the robot. Its responsibilities include reading wheel encoder signals using interrupt service routines, acquiring sensor data from the IMU, computing wheel velocities, updating position estimates, executing control and navigation algorithms, and managing wireless communication. Meanwhile, the Arduino Uno operates as a dedicated motor controller, receiving wheel speed commands from the ESP32 over an I2C interface and generating the corresponding motor driver signals.

Together, these operations define the robot's main control loop. At each cycle, sensor data is acquired and processed, the robot's pose is updated, control and navigation commands are computed, and motor commands are issued. The following sections describe each stage of this control loop in detail.

3.2.3 SENSOR DATA ACQUISITION

Sensor data acquisition is performed by the ESP32 and forms the first stage of the control loop. Wheel encoder signals are processed using interrupt service routines to ensure accurate counting of encoder ticks regardless of processor load. To improve timing reliability and reduce interference from other tasks, encoder interrupt handling was assigned to a dedicated ESP32 core, allowing encoder counts to be captured with minimal latency and jitter

Encoder tick counts are accumulated over each control cycle and used to compute wheel rotation and speed. In parallel, inertial sensor data is acquired from the IMU at a fixed sampling rate. The IMU's onboard digital low-pass filter (DLPF) is configured in software to reduce high-frequency noise in both accelerometer and gyroscope measurements.

3.2.4 ODOMETRY AND POSE ESTIMATION

Following sensor acquisition, the robot's pose estimate is updated. Encoder-derived wheel displacements are converted into incremental changes in position using a differential-drive kinematic model. These incremental updates are applied at each control cycle, allowing the robot's estimated position to evolve smoothly over time.

Heading estimation is performed using a complementary filter that fuses encoder-derived heading changes with gyroscope measurements from the IMU. While early designs used a weighted combination of both IMU and encoder data to estimate orientation, experimental testing showed that gyroscope-based heading estimation was sufficiently accurate on its own under most operating conditions. As a result, the final odometry implementation relies primarily on wheel encoders for distance estimation and the IMU for heading estimation.

Testing demonstrated that this framework produced reasonably accurate position and orientation estimates when the robot was moved by hand or operated under controlled motion.

3.2.5 CONTROL AND NAVIGATION

The robot supports multiple operating modes, each corresponding to a different control objective. Commands received over the Bluetooth interface are used to place the robot into a specific mode, allowing individual subsystems to be used independently during testing.

In DRIVE mode, enabled by sending the command “GO X” to the robot over Bluetooth, the robot seeks a target wheel speed specified by the user. Independent PID controllers regulate the left and right wheel speeds using the wheel speed observed by the encoders. This mode was primarily used to evaluate wheel-speed estimation and motor control performance during straight-line travel.

In TURN mode (“TURN X”) the robot attempts to rotate by a specified angle. In this mode, heading error is regulated using a proportional-only controller acting on the estimated orientation. This simplified control strategy was selected to isolate heading estimation and rotational behavior without introducing additional dynamics from the wheel-speed PID controllers.

In NAVIGATION mode (“TO X Y”), a high-level input–output state feedback linearization (IOSFL) controller is used to drive the robot toward a specified target position. Based on the robot’s estimated pose and the position error relative to the goal, the IOSFL controller generates desired velocity and angular velocity commands. These commands are converted into individual wheel speed targets and passed to the wheel-speed PID controllers.

A stop command (“STOP”) is also implemented to return the robot to an idle state. This command disables active control outputs and halts motion, allowing safe interruption of autonomous behavior and controlled transitions between modes.

Other auxiliary commands include “Zero” (sets pose to {0,0,0}) and “Cal” (performs an IMU calibration), as well as commands for setting the k constants within the PID and IOSFL controllers (“set PID kx x” and “set iosfl kx x”).

3.2.6 COMMUNICATION

A Bluetooth-based communication interface implemented on the ESP32 provides a wireless connection for user input and diagnostic monitoring. This interface allows the user to issue motion commands, specify navigation targets, and switch between operating modes in real time.

The communication interface also streams diagnostic data, including estimated pose, wheel speed measurements and targets, and control outputs and constants. This diagnostic capability was used extensively during development to observe system behavior, validate estimation accuracy, and identify limitations in control performance.

For this project, the nRF Connect mobile application (available for free through the Apple App store) was used to communicate with the robot via the Nordic UART BLE protocol.

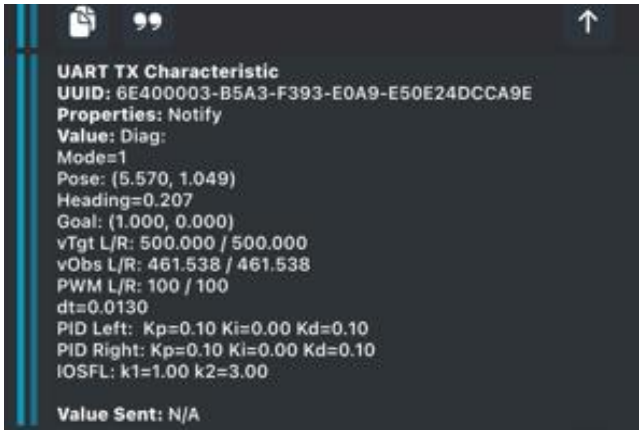

Figure 4. Screenshot of the nRF Connect app showing diagnostic data stream sent from Robot

3.2.7 EXPERIMENTAL TESTING

A series of structured experiments were conducted to evaluate the performance of the robot's sensing, estimation, and control subsystems. Testing was performed in stages, beginning with manual motion tests to validate odometry accuracy, followed by progressively more autonomous control experiments. This approach allowed individual subsystems to be evaluated in isolation before being combined into higher-level behaviors.

Initial testing focused on validating the accuracy of the odometry and pose estimation framework independently of motor control effects. To achieve this, the robot was moved manually while sensor data were recorded and the estimated pose was observed.

A straight-line test was performed in which the robot was manually pushed forward by 1 meter. The estimated displacement produced by the odometry system was compared against the known travel distance. This test was repeated multiple times to assess consistency.

An arc motion test was also conducted, in which the robot was manually moved along a curved path consisting of approximately 1 meter of forward travel combined with a lateral displacement of approximately 0.5 meters. The robot was oriented to finish the motion at an approximate heading of 90 degrees relative to its starting orientation. The final estimated pose was compared to the expected endpoint.

In addition, manual turn-in-place tests were performed to evaluate heading estimation accuracy. The robot was manually rotated by approximately 90 degrees and 180 degrees, and the estimated heading change was recorded.

Across these manual tests, the odometry system produced consistent and reasonably accurate estimates of position and orientation. These results indicate that the sensing and estimation pipeline was functioning correctly under controlled motion conditions.

Point-to-point navigation tests were conducted to evaluate the integrated performance of state estimation, control, and navigation. In these tests, the robot was commanded to navigate to a specified target position using the IOSFL-based navigation mode ("TO X Y"). The robot's estimated pose, control commands, and observed motion were monitored to assess qualitative navigation behavior and identify limitations in autonomous convergence.

Figure 5. (Left to Right) Robot at starting point of test setup; Robot at ending point of straight line test; Robot at ending point of arc motion test

4. RESULTS

This section presents the experimental results obtained from the testing procedures described in Section 4. Results are divided into two categories: quantitative results from manual motion tests used to validate odometry and pose estimation, and qualitative observations from autonomous control and navigation experiments.

In the straight-line tests, the robot was manually pushed forward by approximately 1 meter from a stationary starting position. The odometry system produced distance estimates that functionally matched the expected travel distance across repeated trials. Errors remained small and consistent, indicating reliable encoder-based distance estimation under controlled conditions. Results from these trials are summarized in Table 1:

Straight Line Test, 1 Meter Target Pose: (x = 1.00m, y = 0.00m, Θ = 0.00 rad)			
	Observed x (m)	Observed y (m)	Observed Θ (rad)
Trial 1	0.931	0.008	0.007
Trial 2	0.895	0.033	0.015
Trial 3	0.953	0.04	0.038

Table 1. Straight Line Test, 1 Meter

During curved-path tests, the robot was manually moved along an arc consisting of approximately 1 meter of forward travel and 0.5 meters of lateral displacement, finishing at a heading approximately 90 degrees from the initial orientation. The estimated final position and orientation closely matched the expected endpoint. These results demonstrate that the odometry system was able to track combined translational and rotational motion accurately when encoder and IMU data were integrated. Quantitative results are summarized in Table 2.

Arc Motion Test, 1 meter by 0.5 meters Target Pose: (x = 1.00m, y = 0.50m, Θ = 1.571 rad)			
	Observed x (m)	Observed y (m)	Observed Θ (rad)
Trial 1	1.023	0.0416	1.595
Trial 2	1.024	0.0424	1.548
Trial 3	1.045	0.0356	1.558

Table 2. Arc Motion Test, 1 meter x 0.5 meters

Manual turn-in-place tests were conducted for approximately 90-degree and 180-degree rotations. The estimated change in orientation closely matched the expected rotation angles in both cases.

These results indicate that IMU-based heading estimation was accurate and repeatable under manual rotation, supporting its use as the primary source of orientation information in the odometry framework. Results from these tests are summarized in Table 3.

`` Manual Turn-In-Place Test		
	Target Heading (rad)	Observed Heading (rad)
Trial 1	1.571	1.570
Trial 2	1.571	1.578
Trial 3	1.571	1.554
Trial 4	3.142	3.105
Trial 5	3.142	3.137

Table 3. Manual Turn-In-Place Results

Autonomous turn-in-place experiments were conducted using the rotational control mode, which commands the robot to rotate by a specified angle based on heading error.

While the robot was able to initiate rotation at higher command values, turn accuracy was inconsistent. Friction, wheel slip, and asymmetric motor behavior resulted in overshoot, undershoot, or stalled motion depending on the commanded angle and surface conditions. These effects were particularly pronounced at small commanded angles, where the control effort was insufficient to overcome static friction.

Because rotational motion could not be produced in a smooth or repeatable manner, turn accuracy could not be evaluated quantitatively. These results indicate that mechanical limitations, rather than heading estimation error or control logic, dominated rotational control behavior.

5. DISCUSSION

While the sensing and estimation subsystems performed reliably under controlled conditions, overall system performance during autonomous operation was dominated by mechanical limitations. The most significant of these was the presence of a large motor dead zone, in which PWM commands below approximately 80 failed to produce any measurable wheel motion. As a result, a deadband was required to initiate movement during autonomous control.

During turning maneuvers, this limitation became more severe. Rotational motion requires differential wheel speeds and increased torque to overcome static friction and wheel slip. In practice, PWM values sufficient for straight-line motion were often insufficient to initiate turning, requiring even higher command values. Raising the deadband to accommodate turning improved motion initiation but further reduced the effective resolution available to the controller.

This tradeoff between motion initiation and control resolution significantly constrained both wheel-speed PID control and higher-level navigation. With a large deadband in place, small corrective commands—particularly those generated near a target position—fell entirely within the nonresponsive region of the drivetrain. As a result, fine adjustments could not be realized, leading to stalled motion rather than gradual convergence to the target.

Additionally, using large PWM values to overcome friction and initiate motion introduced other issues related to power delivery. High motor currents drawn during extreme commands caused noticeable voltage drops, which intermittently disrupted communication and control stability. These effects further limited the usable control range and made sustained autonomous operation unreliable.

Together, these factors show a mismatch between the assumptions of the kinematic control model and the physical behavior of the drivetrain. The unicycle and differential-drive models assume smooth, continuous mapping between commanded velocities and realized motion. In this system, limited PWM resolution, motor torque characteristics, wheel slip during turning, and power supply constraints violated these assumptions, preventing effective realization of low-speed and fine-control behaviors.

6. FUTURE IMPROVEMENTS

The results of this project indicate that future improvements should prioritize drivetrain and motor performance before further refinement of control and navigation algorithms. Replacing the existing motors with higher-quality units featuring lower static friction, reduced dead zones, and more predictable torque characteristics would significantly improve low-speed controllability. This would enable more meaningful tuning and evaluation of wheel-speed PID controllers and higher-level navigation strategies such as IOSFL.

With this in place, future work could focus on more advanced state estimation methods. More sophisticated methods, such as Luenberger observers or extended Kalman filters (EKF), could be implemented to better fuse encoder and inertial measurements. These techniques would allow uncertainty to be modeled explicitly and could improve robustness under noisy or partially slipping conditions.

Beyond navigation and control, additional sensing capabilities could be added to support higher-level autonomy. Integrating object detection or obstacle avoidance using vision or range sensors would enable the robot to perceive its environment and react to external constraints. These capabilities would allow the project to return to its original objective of autonomous parking behavior, extending the system beyond dead-reckoning-based navigation.

7. CONCLUSION

This project investigated position estimation and control for an autonomous mobile robot. Encoder-based distance estimation combined with IMU-based heading estimation produced accurate and repeatable pose estimates under controlled conditions. However, extending these results to autonomous control revealed significant limitations due to motor dead zones, friction, wheel slip, and power constraints, which prevented reliable execution of wheel-speed PID control and IOSFL-based navigation. The system architecture and results provide a solid foundation for future improvements in control, state estimation, and autonomous behavior.

8. REFERENCES

[1] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.

[2] Y. Liu, S. Wang, Y. Xie, T. Xiong, and M. Wu, “A review of sensing technologies for indoor autonomous mobile robots,” *Sensors*, vol. 24, no. 4, p. 1222, 2024, doi:10.3390/s24041222.